Problem Sets

HW1. Basics: Circuits, Functions, and Representations

Note: I have put key words in **bold** in the text below. You may look these up (e.g. in the reading, or on Wikipedia) for help or context on a given problem.

I. Combinatorial Circuits

Design **combinatorial circuits** with the following input-output profiles:

1. **NAND**

IN 1	IN 2	OUT
0	0	1
0	1	1
1	0	1
1	1	0

2. Reconstructing OR

IN 1	IN 2	OUT
0	0	0
0	1	1
1	0	1
1	1	1

<< For this problem, do not use OR-gates.

DeMorgan's Law is useful here!

3. Splitting outputs

IN 1	IN 2	OUT1	OUT2
0	0	0	1
0	1	0	1
1	0	0	1
1	1	1	0

4. **XOR** (**exclusive OR**)

IN 1	IN 2	OUT
0	0	0
0	1	1
1	0	1
1	1	0

5. Combining circuits

IN 1	IN 2	OUT1	OUT2
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

<< You can box your XOR circuit from Problem 4 and reuse it here.

Hint: you might think about the circuits here as systematically analogous to equations in propositional logic, and the input-output tables as analogous to truth-tables. In many cases, you can use your knowledge of logic to work backwards to find the appropriate circuit.

II. Functions

6. Consider the function $m(\cdot)$ defined by the following **equation**:
   for any integers x, y, and z: $m(x, y, z) = 3 + (x \bullet y \bullet z)$
   a. Define the function in English, with no mathematical symbolism, using the terms "function", "argument", "value", "apply", and "return" correctly.
   b. Describe the function with another English sentence using the phrase "maps ... to".
   c. How many **places** does this function have?
7. Consider the function $k(\cdot)$ that takes two integers x and y as arguments, and returns the **successor** of their product as its value. Define this function with an equation.
8. This is a problem about **function composition**. Let $s(\cdot)$ be the 1-place **successor function**. Let $a(\cdot)$ be the 2-place **addition function**. Suppose that $p(\cdot)$ is a 2-place function composed of successor functions and addition functions. Given that $p(1, 2) = 5$, provide two possible definitions of $p(\cdot)$. Your two definitions should define different functions: functions that agree on the arguments (1, 2) but disagree on some other arguments.
9. Consider the function $j(\cdot)$ that maps two integers x and y to their product, so long as both x and y are between 0 and 2, inclusive. For any other arguments, it is undefined.
   a. Define $j(\cdot)$ with an equation.
   b. Define $j(\cdot)$ with a table.

   This is a problem about **partial functions**.
10. The **Boolean numbers** are 0 and 1. A **Boolean function** is a function which has only Boolean numbers as arguments and values.
    a. Using an equation, define a 1-place Boolean function which maps 1's to 0's, and 0's to 1's.
    b. Using an equation, define a 1-place Boolean function which maps any Boolean number to itself. This is called the **identity function**.
    c. Using an equation, define a 1-place Boolean function which maps any Boolean number to the number one. This is called a **constant function**.

III. Representations

11. Represent the numbers zero through ten in **binary**.

12. Invent a new **base-6 counting system**, with all new symbols for the 6 digits (including zero!). List your basic symbols and their meanings. Then represent the number thirty-two in this system.

13. a. Add the binary number 1111 to the binary number 1.
    b. Add the binary number 1011 to the binary number 11001.
    Use one of the methods of addition from the Petzold reading.

14. Define a function *f* which maps the symbol "@" to the number zero and the symbol "#" to the number one. Use a table or an equation.

15. Suppose we use the symbols in Problem 14 to form a base-2 counting system. Let's call these symbols "marks". A mark is either @ or #. Using an equation, define a 3-place function *g* which takes any three marks in a row and maps them onto the numerical value of that string of marks in that base-2 system. You will have to use the function in Problem 14 as part of the definition.

    So, for example, your function should predict that
    $g(@, @, \#)$ = one $g(@, \#, @)$ = two and so on…
    Your answer should take the form:
    "for any series of three marks x, y, z: $g(x, y, z)$ = …."

Hint: there are two things going on in Problem 15. It asks you to use your knowledge of functions to define the numerical values of symbols in a base-2 counting system. The method of computing binary values described by Petzold will be of help here. But it is also asking you to keep track of the difference between a *symbol* and its *content*, in this case the difference between "#" and *the number one*. Remember that only numbers, but not symbols, can be added and multiplied! At a deeper level, this problem asks you to give a partial **semantics** for a 3-place representational system. The science of formal semantics extends this kind of analysis to the domain of language.

Challenge problem (optional): define a function which maps strings of 1's and 0's of *arbitrary length* to their numerical values. This is the general problem of semantics for binary.

IV. Synthesis

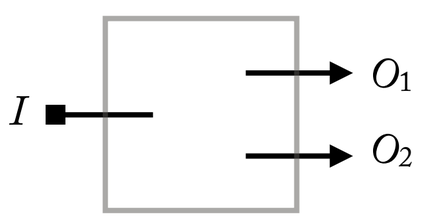
16. Let $tal(\cdot)$ be a function which maps strings of tally 1's and 0's to their numerical values. For example, $tal(0)$=zero, $tal(1)$=one, $tal(00)$=zero, $tal(01)$=one, $tal(11)$=two, and so on.

    Consider a machine *M* which takes as input a single atomic symbol *I*, and produces two-place symbol O_1O_2 as output (in the form of two atomic symbols O_1 and O_2), such that, for any input, $successor(tal(I)) = tal(O_1O_2)$. For example, 1 → *M* → 11. Design a combinatorial circuit for machine *M*, with the schematic form below.

17. Let $bin(\cdot)$ be a function which maps strings of binary 1's and 0's to their numerical values. For example, $bin(0)$=zero, $bin(1)$=one, $bin(00)$=zero, $bin(01)$=one, $bin(10)$=two, $bin(11)$=three, and so on.

    Consider a machine *N* which takes as input a single atomic symbol *I*, and produces two-place symbol O_1O_2 as output (in the form of two atomic symbols O_1 and O_2), such that, for any input, $successor(bin(I)) = bin(O_1O_2)$. For example, 1 → *N* → 10. Design a combinatorial circuit for machine *N*, with the schematic form below.

Hint: these two problems look complex, but they are just asking you to apply the intuitive concept of **computation** we have been discussing in the course so far. First figure out what table of input and output symbols would satisfy the equations given. Only then solve for the underlying circuit.



Schematic for *M* and *N*

HW2. Computing with CCs

10.10.25

I. CC Arithmetic

Starting with **the three basic logic gates only**, design CCs that compute the following functions. At any stage you may define a new boxed circuit from a previously established circuit or logic gate. Useful circuits to define in this way include **All-1**, **XOR**, **half-adder**, and (if you wish) **full-adder**. You will also want to introduce boxed circuits for specific mathematical functions as the need arises.

1. Half adder
2. +1 [0-15] B
3. +2 [0-15] B
4. +3 [0-15] B
5. $2x$ [0-15] B
6. $2x$+1 [0-15] B
7. 2(x+1) [0-15] B (only use *one* sub-part which computes +1)
8. x+y [0-15] B (i.e. addition, bounded to 0-15 for each argument)
   Reference**:** Petzold (2000) "A binary adding machine."
9. For any integer n, in principle, could you design a CC that computes +1 [0- n] B? Why or why not? Short paragraph.
10. Could you design a CC that computes +1 B (i.e. unbounded)? Why or why not? Short paragraph.

Challenge problem: prove the following: for any numerical function f and any integer n, there is a CC which computes f[0 - n] B.

On binary addition

Hint 1: Use the grade school algorithm for long addition.

Start by thinking of the problem in terms of long addition, e.g.:

	1	1	Input 1
+	1	0	Input 2
1	0	1	Output

Then set up your machine in the same format:

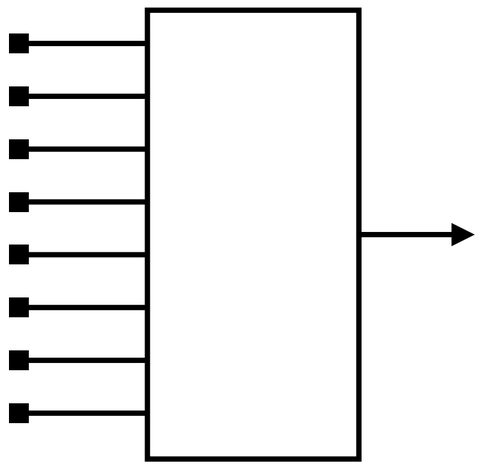
	Digit 4	Digit 3	Digit 2	Digit 1	
	☐	☐	☐	☐	Input 1
	Digit 4	Digit 3	Digit 2	Digit 1	
	☐	☐	☐	☐	Input 2
Digit 5	Digit 4	Digit 3	Digit 2	Digit 1	
☐	☐	☐	☐	☐	Output

When filling in the circuit, think of your machine first in terms of columns (e.g. how does the Output Digit 1 relate to the Input Digit 1's?). Then think about how the columns relate to each other. A key idea here is the "carry digit"– remember how this works in standard long addition, and see if you can apply the same idea to the construction of your machine.

Hint 2: Add twice. When you are doing long addition, note that you really have to do basic addition *twice* for each column. First you have to add the digit from the top row to the digit from the bottom row, and then you have to add the carry to the result. (You can also do these in the opposite order.) The same thing is true in binary, a fact you should convince yourself of before trying to solve the problem. You'll find this idea has a direct correlate in the circuit which computes binary addition.

II. CC Perception

For each of the following problems assume a CC that has 8 inputs and 1 output, like the schematic circuit below. Think of these as computing **perception**: they take as their input an array of stimulations (like hitting the retina) and they output a symbol which represents a specific shape or category.



10. **Edge detector.** Design a machine that obeys the following rule: output 1 iff there is a string of *at least* three consecutive 1's anywhere in the input.

11. **Object detector.** Design a machine that obeys the following rule: output 1 iff there is a string of *exactly* three consecutive 1's anywhere in the input.

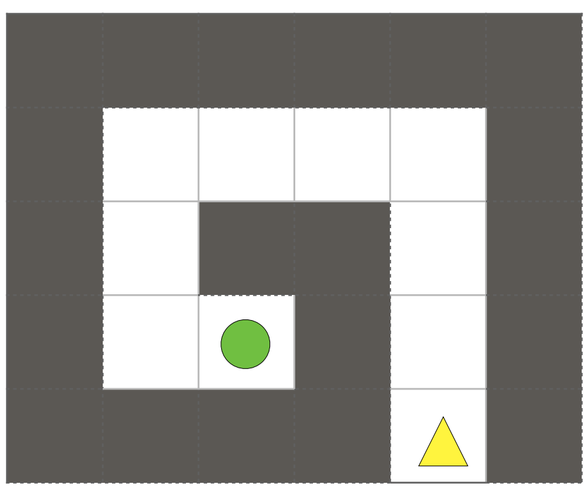
12. **Landmark recognition.** Design a machine that obeys the following rule: output 1 iff the input = 11010111

III. CC Navigation

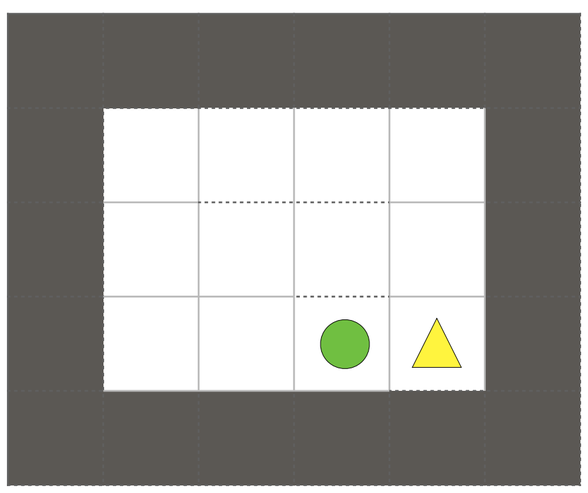
In these navigation tasks, the yellow triangle indicates the turbot's starting position and orientation. Navigation is "completed," and the problem solved, when the turbot *crosses over* the goal. It does not need to stop on the goal. (Think of Pac-Man.)

For each of the following problems, answer the question: can it be completed by a combinatorial circuit embedded in a turbot? If so, give the circuit. If not, explain why in a short paragraph.

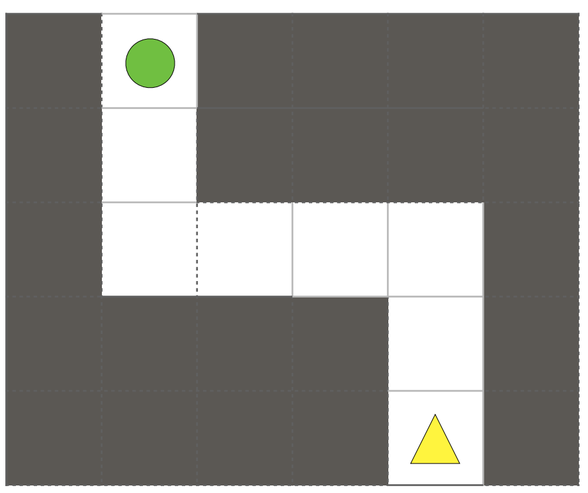
13. **Spiral**



14. **Full circle**



15. **Zig-zag**



IV. Reflection

16. **Representational systems.** Suppose you were designing a calculator for solving a high-stakes problem. (Perhaps "you" are Nature, the "calculator" is the Brain, and the "problem" is Survival.) Would you design it to calculate in binary or in tally? Why? Use specific examples to justify your answer. ~1 paragraph

17. **Embodied computation.** Consider a Turbot like the one in problem 13. What function, if any, does it compute? If it does compute a function, what are the arguments and what is the value of this function? How does this compare to the function computed by the "directions" mode in Google Maps? ~1 paragraph

HW3. Computing with SCs

10.27.25

I. SC Arithmetic

At any stage you may define a new boxed circuit from a previously established circuit. You may assume boxed circuits for **XOR** and **half-adder.** You will also want to introduce boxed circuits for specific mathematical functions as the need arises.

Design SCs that compute the following functions.

1. +1 B
2. +2 B
3. $2x$ B
4. $2x$+1 B [Use one $2x$ circuit and one +1 circuit]
5. $2(x+1)$ B [Use one $2x$ circuit and one +1 circuit]
6. $x+y$ B

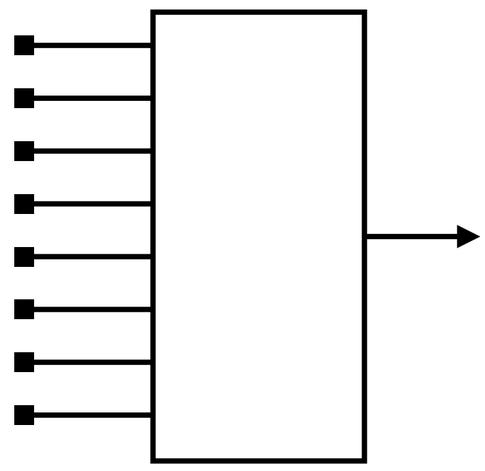
For each of the following functions, is it possible to compute this function with an SC? If so, construct the circuit. If not, explain why in a short paragraph: be as clear and precise as you can.

7. +3 T
8. $2x$ T
9. $x + y$ T
10. $x \bullet y$ B (multiplication)

II. SC Perception

For each of the following problems assume a SC that has 8 inputs and 1 output, like the schematic circuit below. Think of these as computing **perception**: they take as their input an array of stimulations (like hitting the retina) and they output a symbol which represents a specific shape or category.

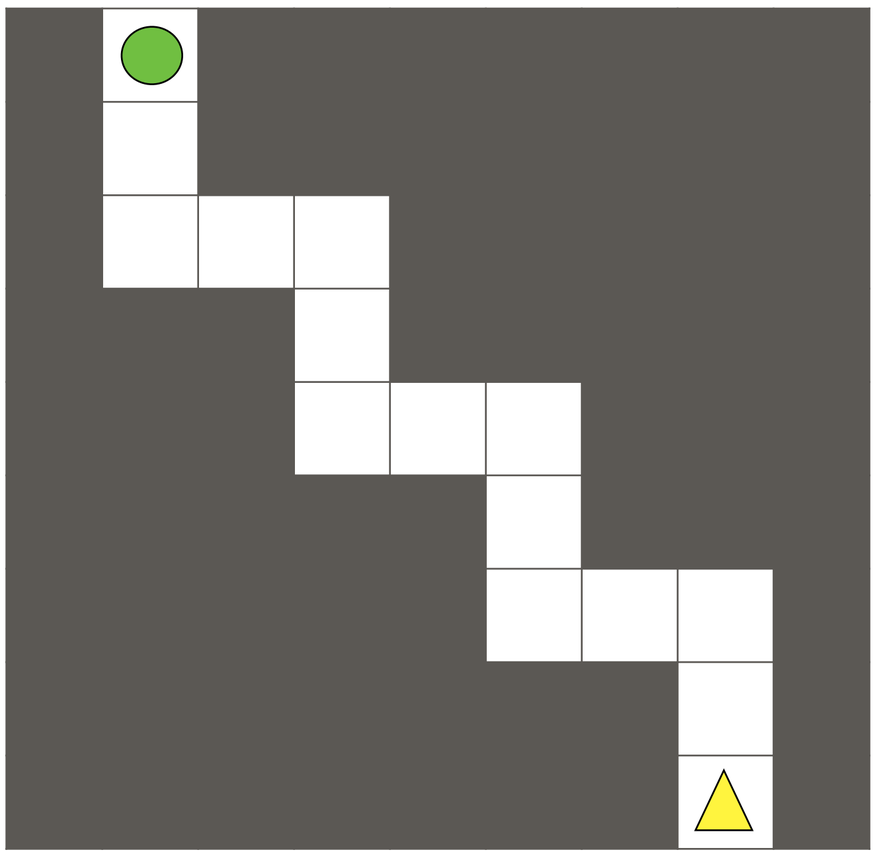
11. **Change detector.** Design a machine that obeys the following rule: output 1 iff the current input differs in any way from the previous input.
12. **Motion detector.** Let an *object image* be a string of exactly three consecutive 1's. Design a machine that obeys the following rule: output 1 iff there is an object image anywhere in the input moving upwards 1 unit per unit of time.



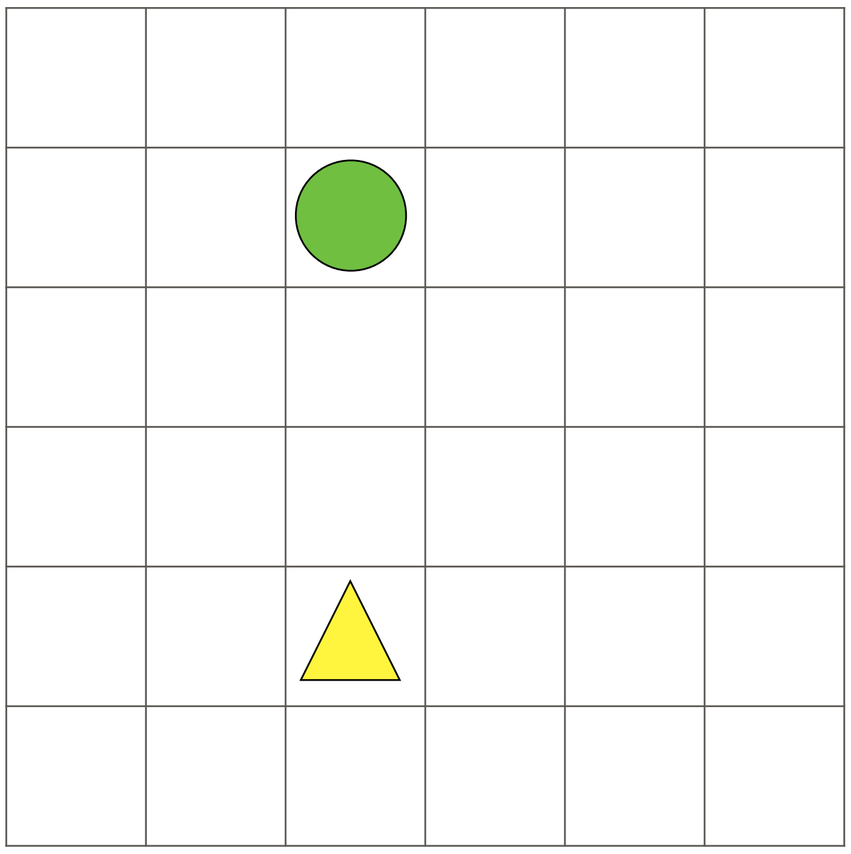
II. SC Navigation

For each of the following navigation tasks: can it be completed by a combinatorial circuit embedded in a turbot? If so, give the circuit. If not, explain why in a short paragraph.

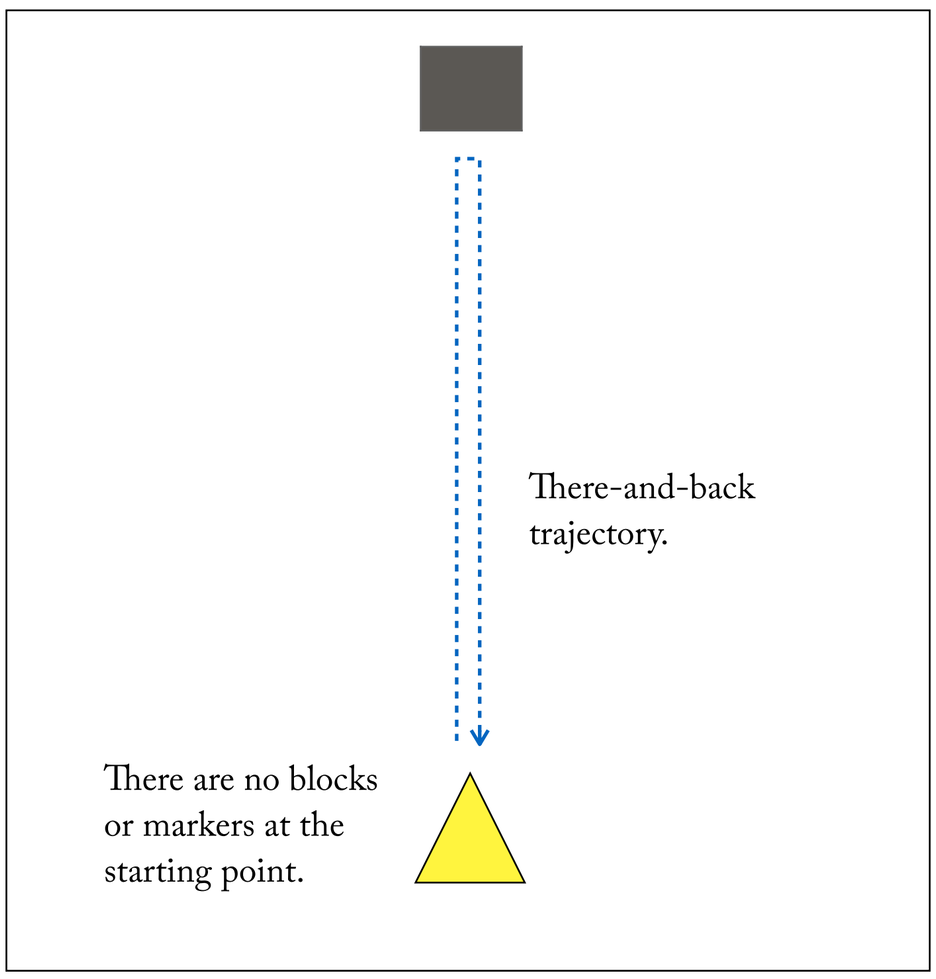
13. **Zig-Zag:** reach the goal and **keep going**.



14. **Three ahead:** reach the goal and **stop**.



15. **Mad Max:** go to a block directly ahead, but an unknown distance away, then return to the starting point and **stop**.



Note 1: SC Turbots have two outputs, one for each wheel.

Note 2: The goal (the green circle) is invisible to the turbot; it reads as an empty block.

III. Reflection

Short paragraph answers.

16. SCs are more computational powerful than CCs: SCs can solve problems that CCs cannot. What physical abilities would you add to an SC if you wanted to make it even more powerful?

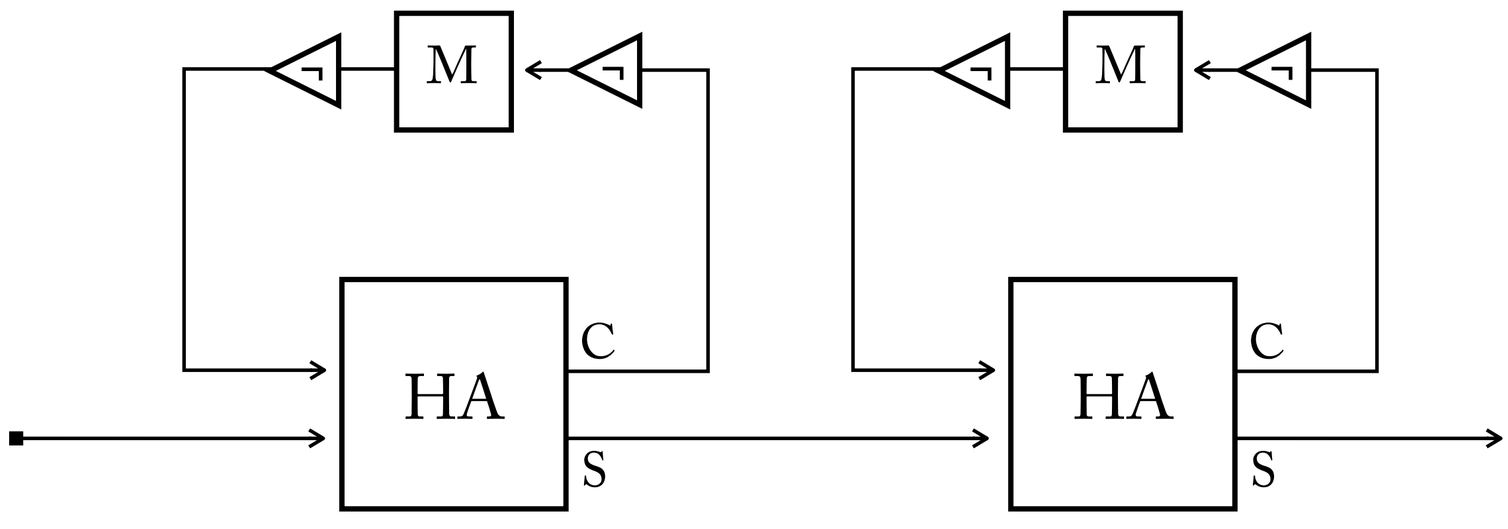
17. Could there be an intelligent being with no memory?

HW4. Computing with FSMs

11.11.25

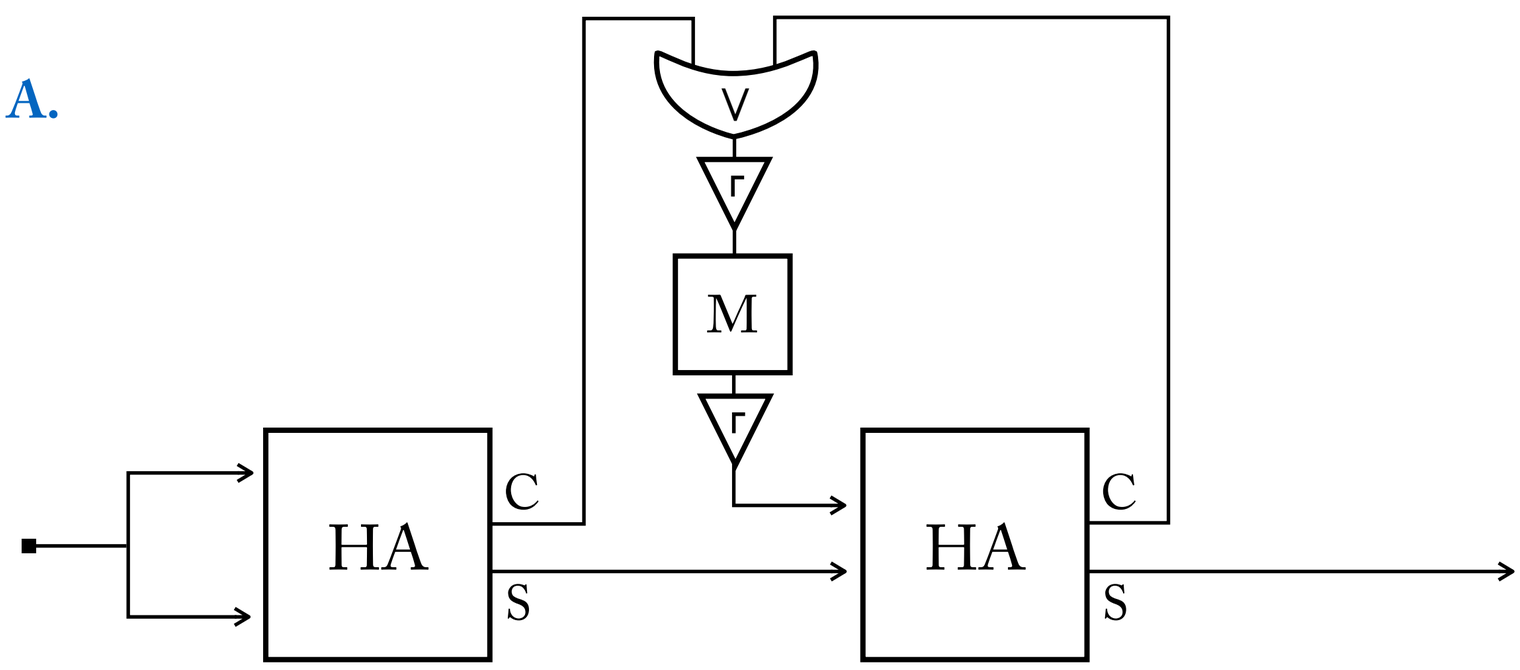
I. State Abstraction

1. **State abstraction.** Construct a state table for the following machine, then convert it to a FSM diagram. What function does it compute in binary?

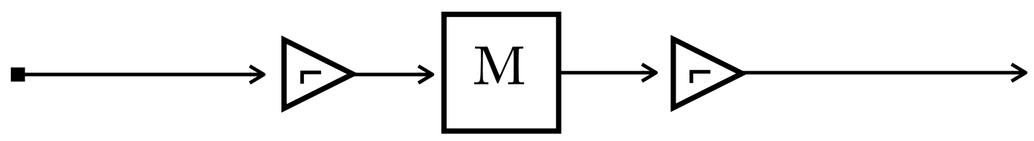


2. **Multiple realizability.** Consider the following two SCs. What function does each compute in binary? Do they implement the same FSM? Why or why not? Show your work.

A.



B.



II. FSM Arithmetic and Beyond

Design FSMs that compute the following functions. (Don't design an SC first to get to your solution.)

3. +1 T

4. +2 T

5. +1 B

6. +2 B

7. +3 B

8. $2x$ B

9. $2x$+1 B

10. $x + y$ B

> **Challenge problem:** prove the following: for any numerical function f and any integer n, there is an FSM which computes f[0 - n] B.

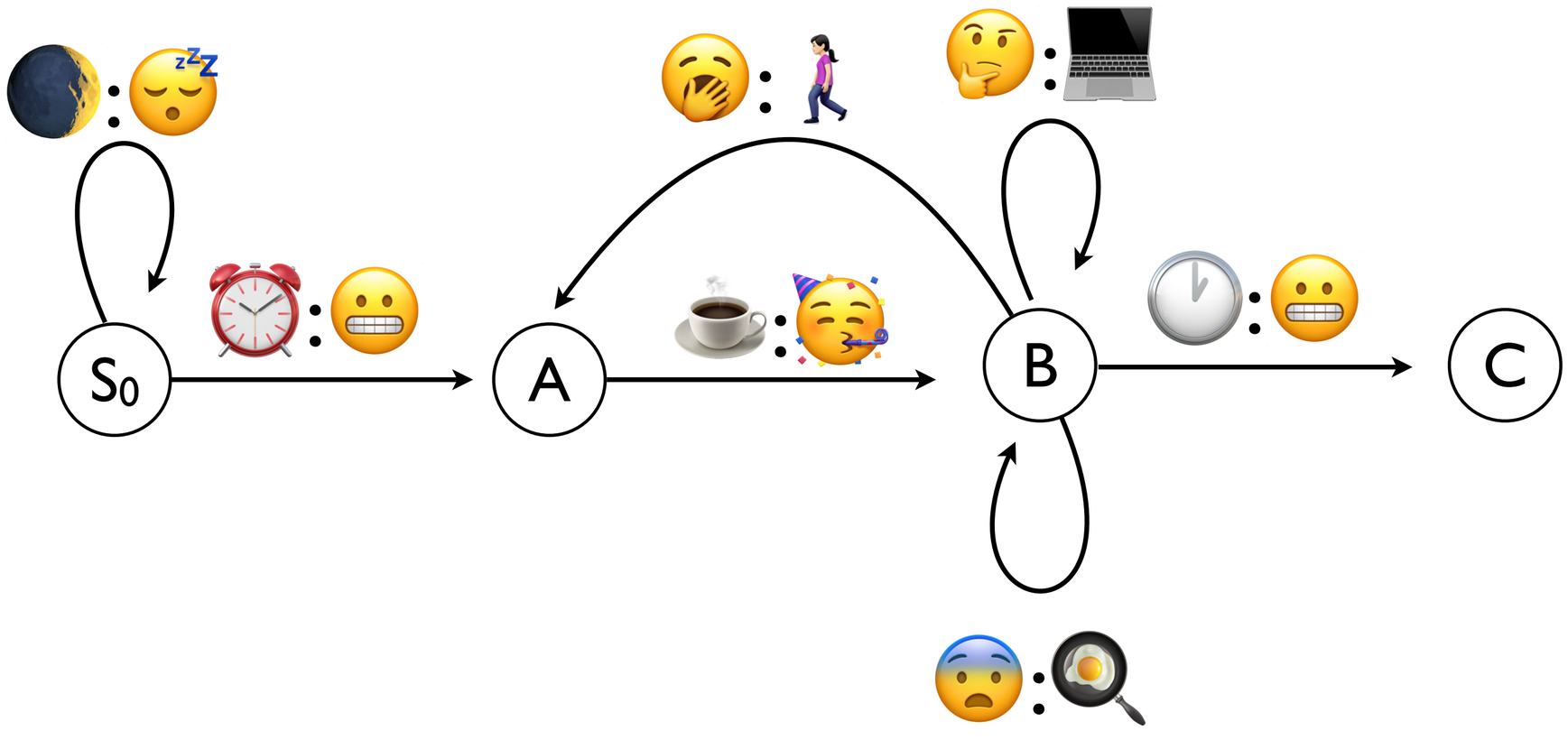
11. **ON/OFF Switch**

    This circuit toggles between outputting 1's or 0's; it switches each time it receives a 1, but keeps the same output if it receives a 0. Start in the off state, where output = 0.

12. **The Daily Grind**

    Design an FSM which captures the cycles and possible paths of your typical morning routine, using only emojis as inputs and outputs.

For example, here is a fragment of my day:



III. FSM Navigation

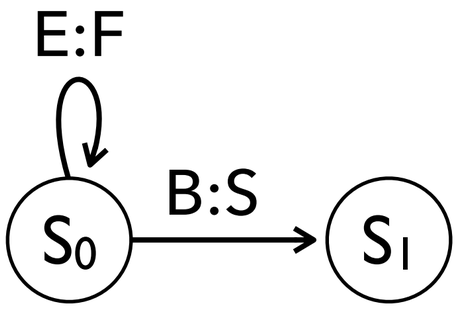
For each of the following navigations tasks, design an FSM which solves it.

Use the following abbreviations:

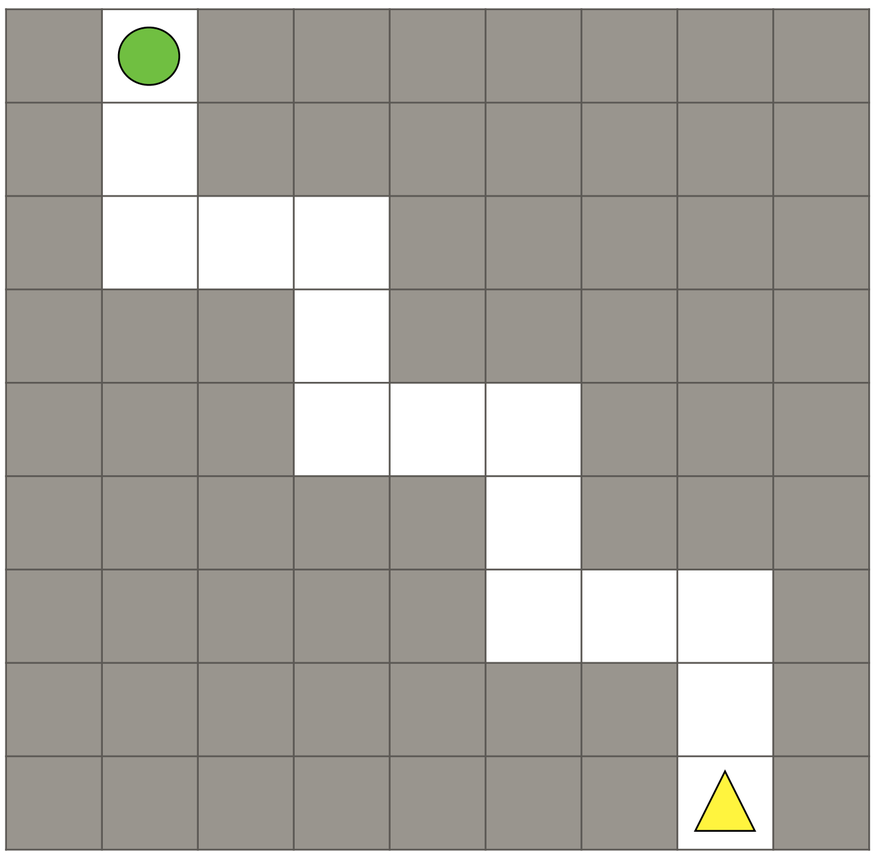
Inputs:
0 = **E** (Empty)
1 = **B** (Block)

Outputs:
11 = **F** (Forward)
00 = **S** (Stop)
10 = **R** (Right)
01 = **L** (Left)

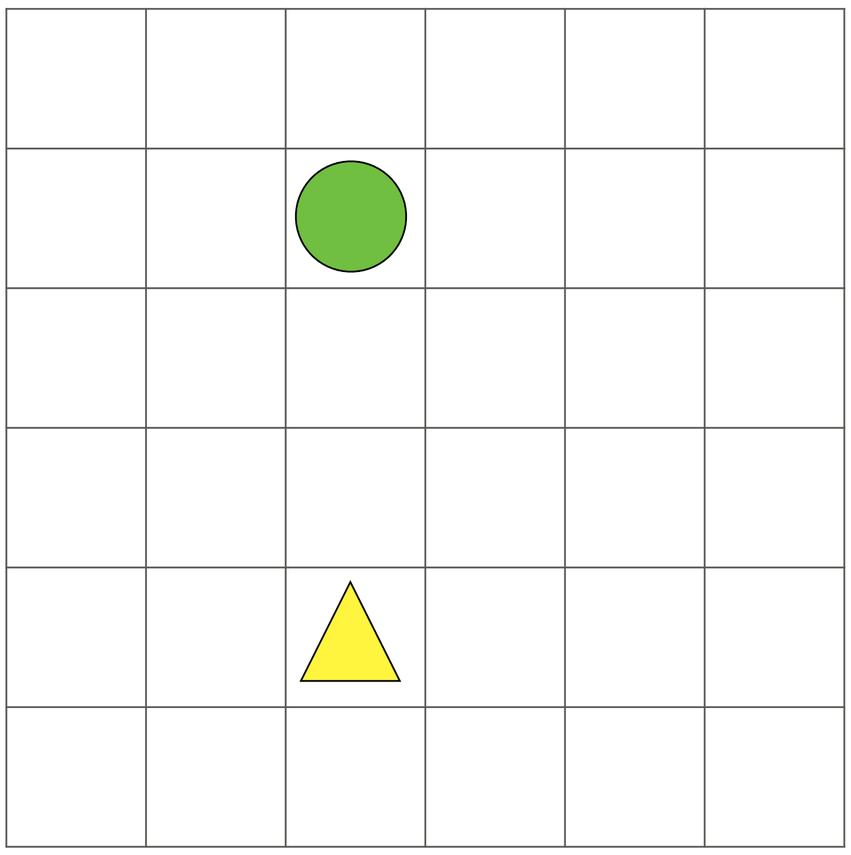
For example, this machine goes forward until it hits a block, then stops:



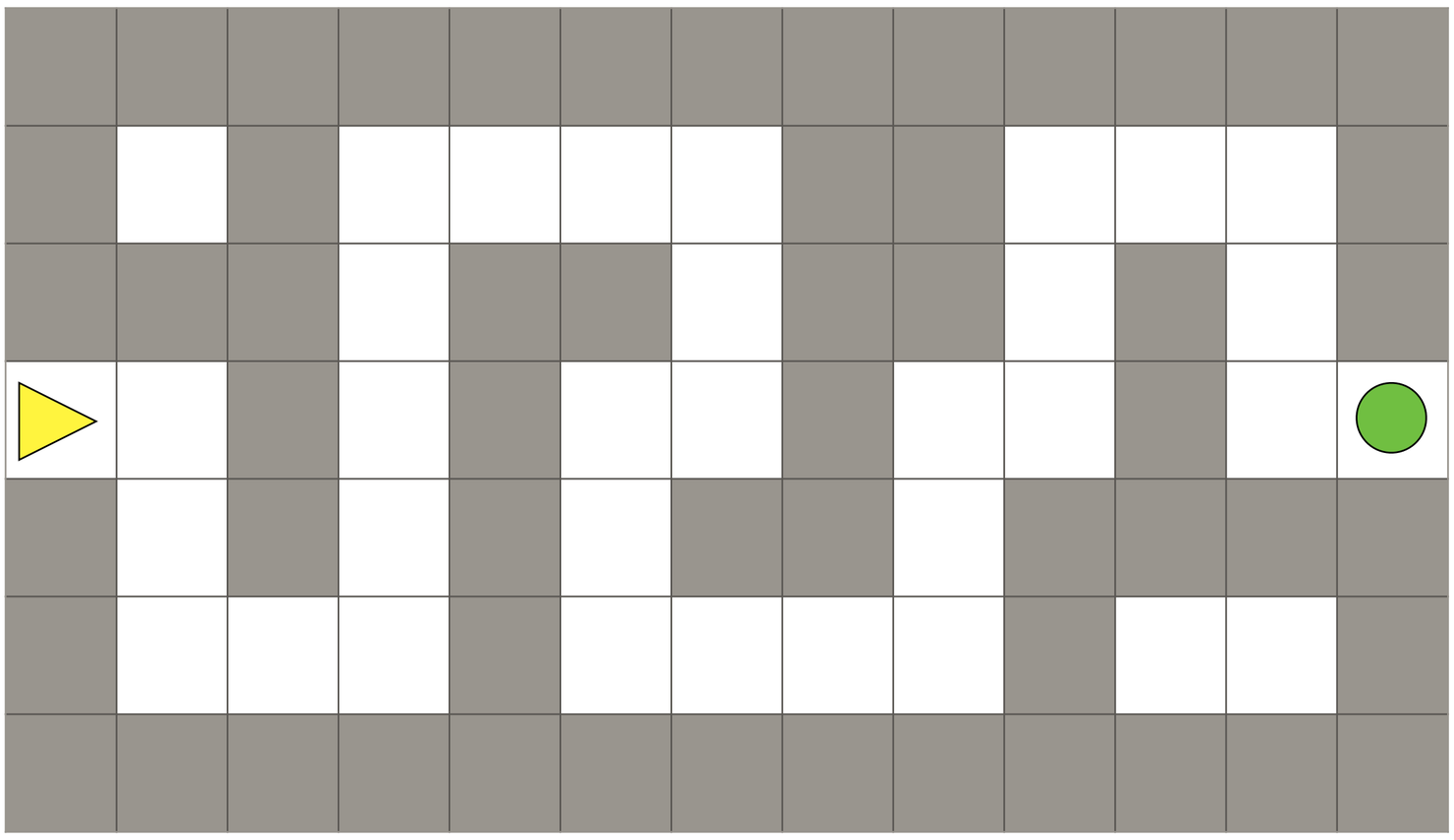
12. **Zig-Zag.** Reach the goal and **keep going**.



13. **Three Ahead.** Reach the goal and **stop**.



14. **Way Finder.** Given *any* non-branching maze, make your way all the way through, and keep going.
A **non-branching maze** is a path that never branches and is never wider than 1-cell across, except on corners where two 1-cell width paths may intersect. Here is an example.*



* This problem was designed by Isaijah Johnson and Jack Schaeffer, Phil 133 W19.

II. FSM Arithmetic and Beyond

Design FSMs that compute the following functions. **Don't** design an SC first to get your solution; **do** think through the FSM directly.

4. +1 T
5. +2 T
6. +1 B
7. +2 B
8. +4 B
9. 2*x* B

Take away problem 9! (Or: replace it with the problem: given the circuit, was is the FSM?)

Caution!

Function composition **does not work** for FSMs. You can't combine two +1 machines to get a +2 machine.

This has to do with the fact that FSM diagrams show states of the *overall* machine, not the flow of information *through* it.

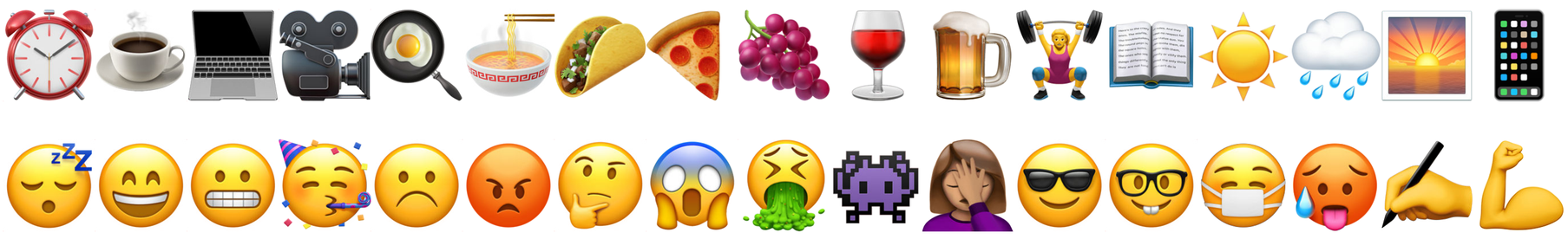
10. **ON/OFF Switch**

This circuit toggles between outputting 1's or 0's; it switches each time it receives a 1, but keeps the same output if it receives a 0. Start in the off state, where output = 0.

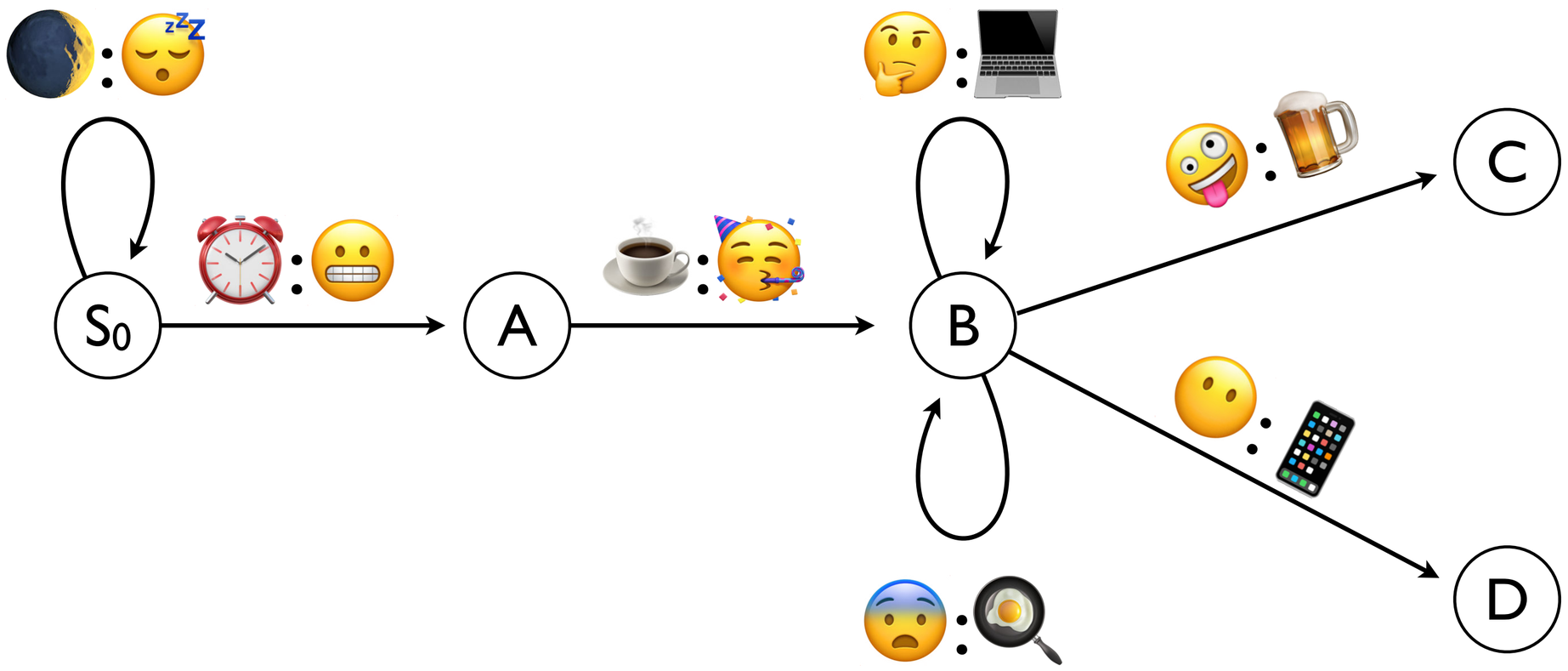
11. **The Daily Grind**

Design an FSM which captures the cycles and possible paths of your typical day during quarantine. Make it one big loop to take you from morning to morning. Feel free to add alternative inputs and outputs as appropriate.

Possible inputs and outputs:



For example, here is a fragment of my day:



HW5. Computing with TMs

2.18.25

Rules

- **Annotation:** all machines must be annotated.
- **Arguments:** Tally problems are defined on arguments $x \geq 1$; for binary problems, $x \geq 0$.
- **Blocks:** blocks are define differently for tally and binary.
  - Tally block = a single block of 1's.
  - Binary block = a block of 0's or 1's, flanked by *s.
- **X- and Y-Blocks:**
  - For 1-place functions, the tape starts with a single block.
    For 2-place functions, the tape starts with two blocks.
  - When a problem involves two arguments x and y:
    - The x-block is on the **right**, and the y-block is on the **left.**
- **Standard Position.**
  - All problems begin in standard position, and must return to standard position.
    - Tally SP = the right most cell of the right most block.
    - Binary SP = the right most * of the right most block.
- **Final configuration:**
  - All problems must end in standard position.
  - All problems must end with a well-formed output: a single block.
- **Boxes**
  - You can use a boxed version of any problem you have solved in an earlier problem, provided it satisfies the criteria for boxing.

I. TM Arithmetic: Tally

Assume $x,y \geq 1$.

1. $x+1$ T

2. $x+3$ T.
   Use your solution from (1).

3. $3x$ T.
   Use your solution from (2).

4. $x+y$ T.
   Use your solution from (1).
   For this problem, do not assume that the input blocks will be only separated by 1 cell. They may be separated by any number of cells.

5. $3(x + y)$ T.
   Use your solutions from (3) and (4).

6. $x + 3y$ T.
   Use your solutions from (3) and (4).

II. TM Arithmetic: Binary

Assume $x,y \geq 0$. The following problems have some arbitrary constraints built-in to make them simpler.

7. $x+1$ B
   • Assume the input block will always have an extra 0 in the left-most cell.

8. $x - 1$ B
   $f(x) = x - 1$ if $x \geq 1$
   $f(x) = 0$ if $x = 0$

9. $x + y$ B
   Use your solutions from (7) and (8).
   • Assume that y-block contain enough 0's padding the left side initially so that the entire block may contain the binary representation of $x+y$.
   • Don't forget to "clean up" your tape at the end of computation.

III. Algorithmic Design

10. Multiplication: $x \bullet y$

    Using a combination of flow charts, prose, and "snapshots" of the tape, describe a step-by-step *strategy* for computing multiplication. This should not be a complete Turing Machine. (That is a difficult and laborious problem.) But it should be sufficiently detailed that you think someone *could* build a Turing Machine from your instructions, if they knew what they were doing. Your choice of Tally or Binary.

11. Multi-function machine

    Design a machine which can perform different functions depending on the user input (at least successor, addition, multiplication), similar to a calculator. There are many way to do this, so be creative. Use a combination of flow charts, prose, and "snapshots" of the tape; don't design a completed Turing Machine.

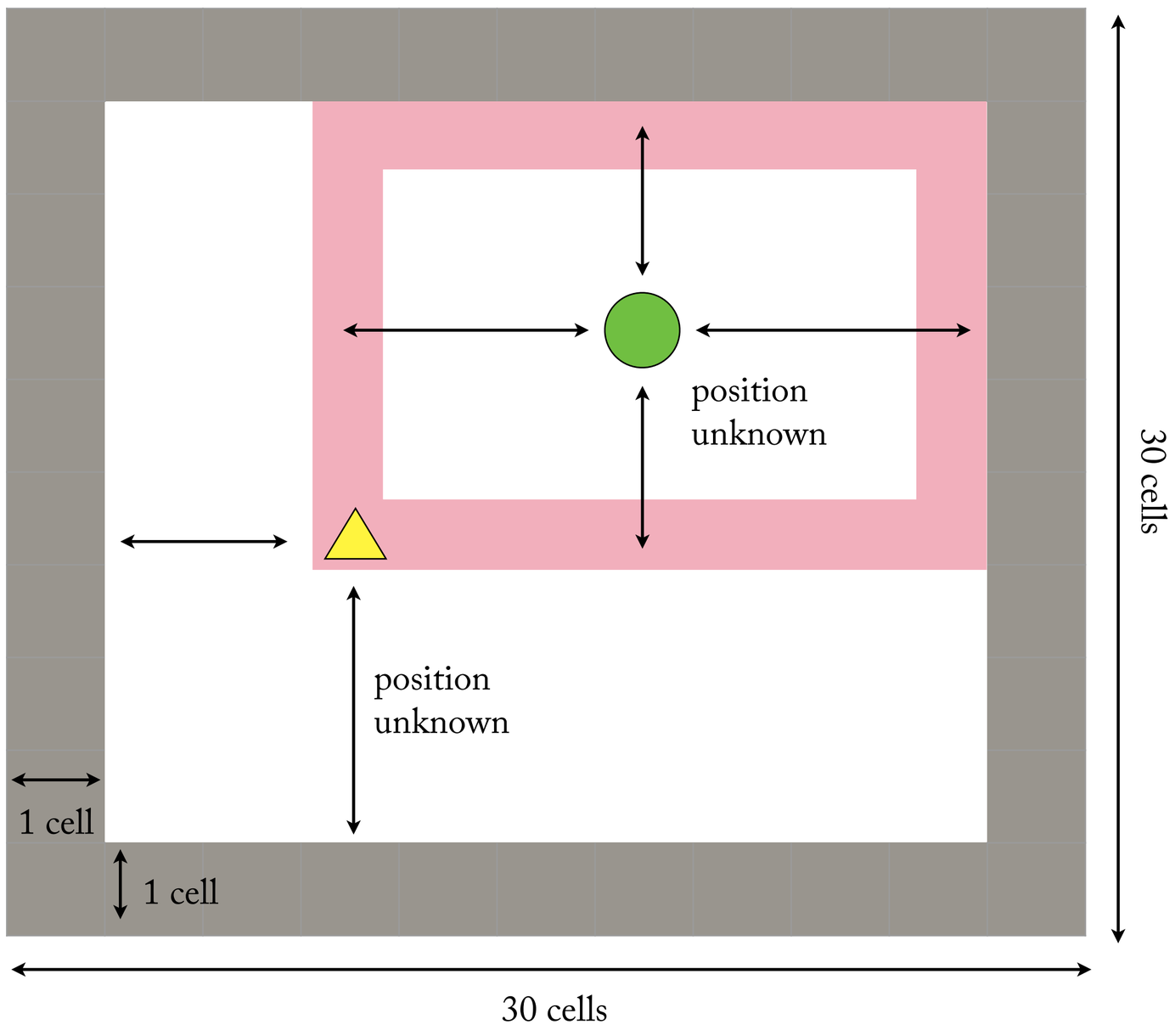
HW6. Navigation with Turbots

2.18.25

The Desert Ant

Goal: Search until you find food (and pass over it), then return to where you started as quickly as possible. You may choose any initial configuration for the tape.

- **Assumption 1:** the food will always appear in the NE quadrant relative to the Turbot's starting position (and not at the same position as the Turbot itself.)
- **Assumption 2:** the world is 30 X 30 cells, surrounded by a wall.
- **Constraint:** use a maximum of 20 cells on the Turing Machine tape.
- **Unknowns:** the starting position of the Turbot in the world is unknown, the position of the food within the NE quadrant of the Turbot is also unknown.



1. Flow-chart.

Design a high-level flow-chart that describes your solution to the Desert Ant Problem. Be sure to describe both the design of the machine and the configuration of the tape.

2. Turbot

Construct a TM that follows flow-chart in in (1) and solves the Desert Ant Problem. Use modular boxed functions when you can. Label all the major parts of your design according to their function.

For very similar functions, it is sufficient to provide one solution, and indicate the obvious change to the other. (For example, if you have one sub-part that handles left turns, and a symmetrical one that handles right turns, you can just design one of these, and describe the other one in words.)

3. The Life Cycle

Suppose that, after solving the Desert Ant Problem, your Turbot must stay at home for a fixed amount of time (say: 20 clock cycles), then return to the food, and the whole cycle must repeat again.

Assuming the food never moves, how would you change your initial design to accommodate this challenge? How would you change the tape? Describe your approach in prose, or with the help of a flow chart, but don't design the actual Turbot.